

Estrategia de *testing*

Gestión de riesgos en *testing*

“Algunas enfermedades son al principio fáciles de curar, pero difíciles de reconocer, pero, a medida que pasa el tiempo, se hacen fáciles de reconocer y difíciles de curar”.

¿Qué es un riesgo?

Un riesgo es un problema que todavía no ocurrió.

Un problema es un riesgo que se manifestó.

En el desarrollo de software, el riesgo clave es “fracasar en hacer el producto correcto”.

Los riesgos tratan sobre eventos posibles del futuro:

- Sin certeza: no se sabe si va a pasar o no.
- Con probabilidad: se puede estimar cuán probable es que pase.
- Con pérdida o impacto: si ocurre, algo malo va a pasar.

El análisis de riesgos es la toma de decisiones informadas bajo incertidumbre:

- ¿Qué puede ir mal?
- ¿Cuál será el impacto?
- ¿Qué se puede hacer?

La administración de riesgos no trata con decisiones futuras, sino con las futuras consecuencias de decisiones actuales.

Los primeros riesgos pueden ser identificados durante las etapas iniciales del proyecto.

Debemos buscar y encontrar los riesgos relacionados con las pruebas del software.

Para ello analizamos: Las fuentes posibles de esos riesgos.

Ejemplos de riesgos posibles:

- Se producen resultados incorrectos.
- Se aceptan transacciones no autorizadas.
- Integridad de archivos.
- No se puede reconstruir el proceso.

- No es rastreable el proceso.
- El servicio al usuario se degrada.
- La seguridad del sistema se ve comprometida.
- Falla de requisitos, requerimientos o pautas.
- Los resultados del sistema no son desplegados.
- Dificultad de uso, problemas de interfaz.
- No mantenible.
- No coincide con los criterios de calidad.
- Interfaz con otros sistemas.
- Eficiencia o *performance*.

Ejemplo de documentación de riesgos en *testing*:

Riesgo	Efectos que puede causar	Acciones preventivas	Acciones correctivas
Inexistencia de un ambiente de <i>testing</i> separado del ambiente de desarrollo.	Inconsistencia entre los resultados de un test y otro. Ineficiencia en la ejecución del plan de <i>testing</i> (retrasos en la entrega final del producto).	Estructurar el ambiente de <i>testing</i> antes del comienzo de esta etapa.	Estructurar el ambiente de <i>testing</i> lo antes posible.
Falta de tiempo para ejecutar casos de pruebas de los diferentes tipos de <i>testing</i> .	Imposibilidad de detectar ciertos errores.	Incorporar en el cronograma tiempos para la realización de los distintos tipos de pruebas.	Priorizar las pruebas. Definir grupos de pruebas en paralelo.
Desarrollo no incremental de los productos de software. El equipo de <i>testing</i> recibe el sistema en grandes bloques y no relacionados entre sí.	Ineficiencia en la ejecución del plan de <i>testing</i> . Retrasos en la ejecución de los tests. "Catarata de errores": Sobrecarga de trabajo en equipo de <i>testing</i> y en equipo de desarrollo.	Considerar en el cronograma de desarrollo la secuencia de implementación de módulos de forma tal que sea incremental, basándose en el diseño de bajo nivel del sistema.	Priorizar las pruebas. Buscar y solucionar primero los errores que bloqueen las aplicaciones.

No disponer de una herramienta de seguimiento de errores.	Problemas para realizar un seguimiento de los errores y observaciones.	Utilizar una base de datos de pruebas para administrar los resultados de las pruebas realizadas.	Priorizar la administración de los resultados de las pruebas.
---	--	--	---

Prueba de unidad

Prueba un módulo en forma independiente.

Generalmente la realiza el área que construyó el módulo.

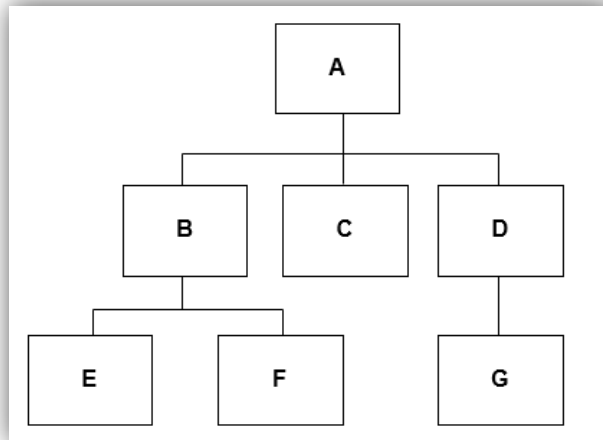
Se basa en la descripción del diseño detallado.

Tipos de prueba:

- Interface.
- Estructuras de datos locales.
- Condiciones límites.
- Caminos independientes.
- Caminos de manejo de errores.

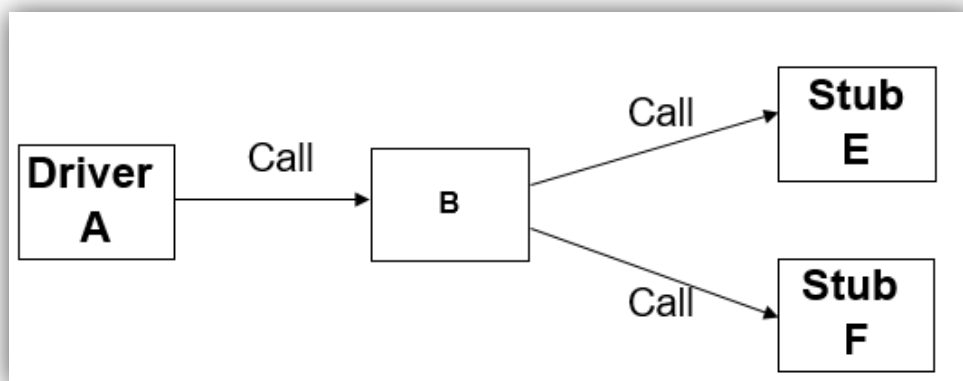
Pruebas de módulos

- El módulo A “usa” a los módulos B, C, D.
- El módulo B “usa” a los módulos E y F.
- El módulo D “usa” al módulo G.



Quiero probar al módulo B de forma aislada – No uso los módulos A, E y F.

- El módulo B es usado por el módulo A:
 - Debo simular la llamada del módulo A al B – *Driver*.
 - Normalmente el *Driver* es el que suministra los datos de los casos de prueba.
- El módulo B usa a los módulos E y F:
 - Cuando llamo desde B a E o F debo simular la ejecución de estos módulos – *Stub*.
- Se prueba al módulo B con los métodos vistos en pruebas unitarias.



Stub (simula la actividad del componente omitido)

- Es una pieza de código con los mismos parámetros de entrada y salida que el módulo faltante, pero con una alta simplificación del comportamiento. De todas maneras, es costoso de realizar.

- Por ejemplo, puede producir los resultados esperados leyendo de un archivo, o pidiéndole de forma interactiva a un testeador humano o no hacer nada siempre y cuando esto sea aceptable para el módulo bajo test.
- Si el *stub* devuelve siempre los mismos valores al módulo que lo llama es probable que esté módulo no sea testeado adecuadamente. Ejemplo: el *Stub* E siempre devuelve el mismo valor cada vez que B lo llama → Es probable que B no sea testeado de forma adecuada.

Driver

- Pieza de código que simula el uso (por otro módulo) del módulo que está siendo testeado. Es menos costoso de realizar que un *stub*.
- Puede leer los datos necesarios para llamar al módulo bajo test desde un archivo, GUI, etc.
- Normalmente es el que suministra los casos de prueba al módulo que está siendo testeado.

Prueba de integración

Es una técnica sistemática para construir la estructura del programa mientras que voy probando las interacciones.

Estrategias de integración

No incremental

- *Big-Bang*

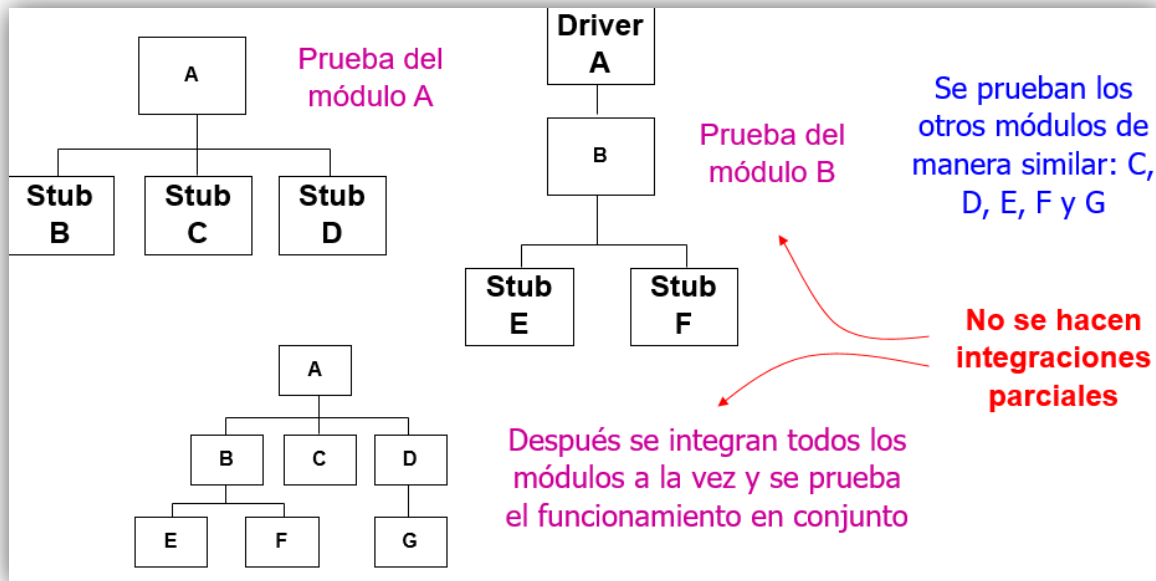
Incrementales

- *Bottom-Up* (ascendente).
- *Top-Down* (descendente).
- Sándwich (intercalada).
- Por disponibilidad.

El objetivo es lograr combinar módulos o componentes individuales para que trabajen correctamente de forma conjunta.

Big-Bang

Se prueba cada módulo de forma aislada y luego se prueba la combinación de todos los módulos a la vez.



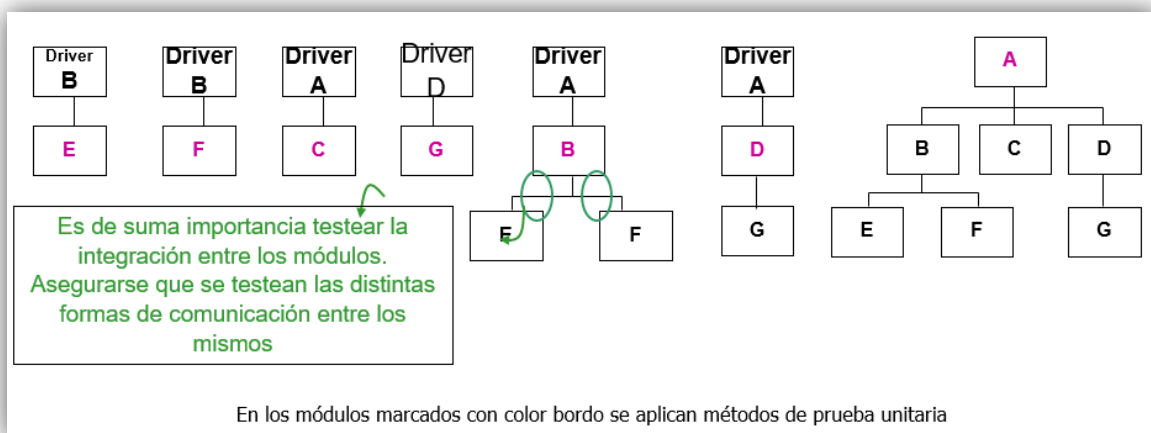
Pros y contras:

Pro	Contras
Existen buenas oportunidades para realizar actividades en paralelo (todos los módulos pueden ser probados a la vez).	Se necesitan <i>stubs</i> y drivers para cada uno de los módulos a ser testeados ya que cada uno se testea de forma individual.
	Es difícil ver cuál es el origen de la falla ya que integré todos los módulos a la vez.
	No se puede empezar la integración con pocos módulos.

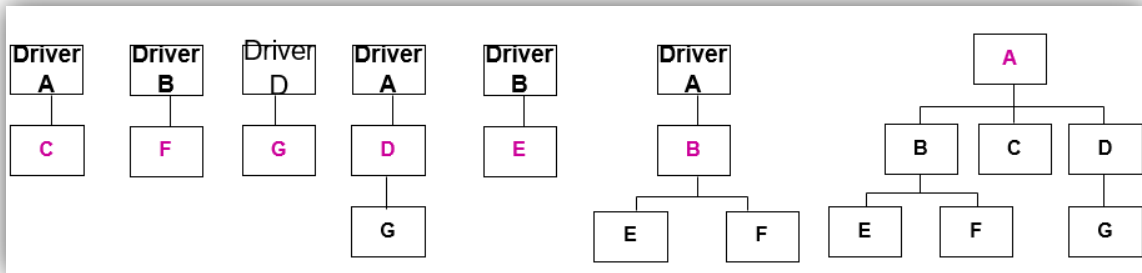
	Los defectos entre las interfaces de los módulos no se pueden distinguir fácilmente de otros defectos.
	No es recomendable para sistemas grandes.

Bottom-Up

- Comienza por los módulos que no requieren de ningún otro para ejecutar.
- Se sigue hacia arriba según la jerarquía “usa”.
- Requiere de *Drivers* pero no de *Stubs*.



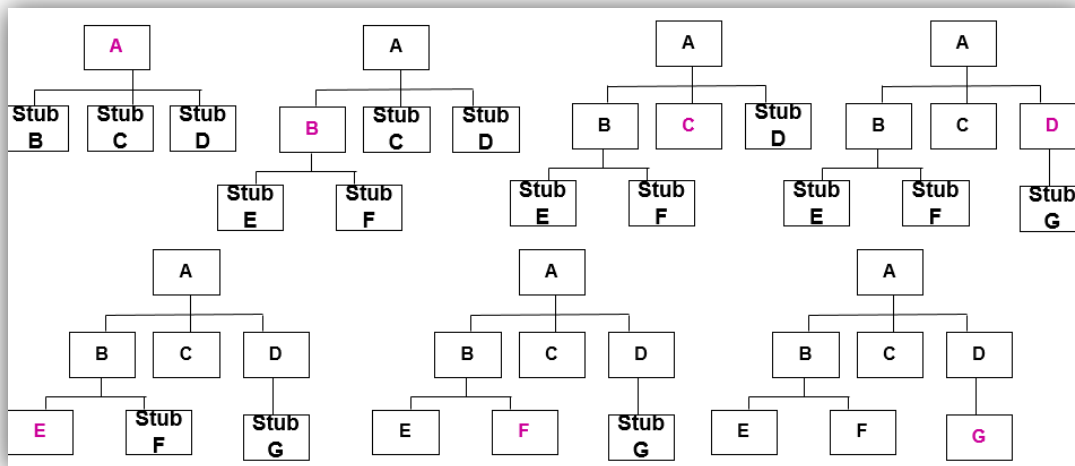
- La regla general indica que para probar un módulo todos los de “abajo” del módulo a probar deben estar probados.
- Supongamos que C, F y D son módulos críticos (módulo complicado, con un algoritmo nuevo o con posibilidad alta de contener errores, etc.) entonces es bueno probarlos lo antes posible.



Hay que considerar que hay que diseñar e implementar de forma tal que los módulos críticos estén disponibles lo antes posibles. Es importante la planificación.

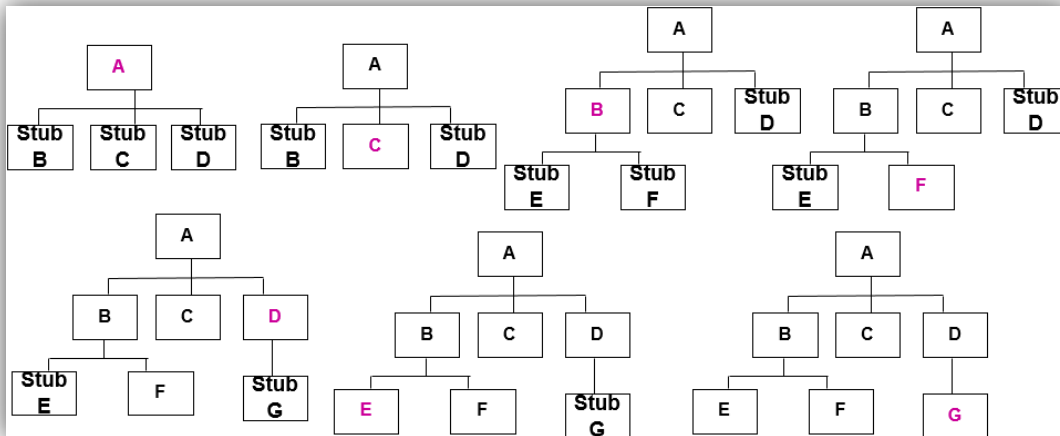
Top-Down

- Comienza por el módulo superior de la jerarquía usa.
- Se sigue hacia abajo según la jerarquía “usa”.
- Requiere de *Stubs* pero no de *Drivers*.



La regla general indica que para probar un módulo todos los de “arriba” (respecto al módulo a probar) deben estar probados.

Supongamos que C, F y D son módulos críticos, entonces, es bueno probarlos lo antes posible.



Otras técnicas:

Sándwich

- Usa *Top-Down* y *Bottom-Up*. Busca probar los módulos más críticos primero.

Por disponibilidad

- Se integran los módulos a medida que estos están disponibles.

Comparación de las estrategias

No incremental respecto a incremental:

- Requiere mayor trabajo. Se deben codificar más *Stubs* y *Drivers* que en las pruebas incrementales.
- Se detectan más tardíamente los errores de interfaces entre los módulos ya que estos se integran al final.
- Se detectan más tardíamente las suposiciones incorrectas respecto a los módulos ya que estos se integran al final.
- Es más difícil encontrar la falta que provocó la falla. En la prueba incremental es muy factible que el defecto esté asociado con el módulo recientemente integrado (en sí mismo) o en interfaces con los otros módulos.
- Los módulos se prueban menos. En la incremental vuelvo a probar indirectamente a los módulos ya probados.

Top-down (focalizando en OO):

- ✓ Resulta fácil la representación de casos de prueba reales ya que los módulos superiores tienden a ser GUI.
- ✓ Permite hacer demostraciones tempranas del producto.
- ✗ Los *stubs* resultan muy complejos.
- ✗ Las condiciones de prueba pueden ser imposibles o muy difíciles de crear (pensar en casos de prueba inválidos).

Bottom-Up (focalizando en OO):

- ✓ Las condiciones de las pruebas son más fáciles de crear.
- ✓ Al no usar *stubs* se simplifican las pruebas.
- ✗ El programa como entidad no existe hasta no ser agregado el último módulo.

Pruebas del sistema

La aplicación de software desarrollada es parte de un sistema más grande.

Pruebo el sistema en un ambiente similar al real.

Hay otras aplicaciones.

Hay hardware y otros equipos.

Tipos de pruebas de sistema:

- Pruebas de recuperación.
- Prueba de seguridad.
- Prueba de resistencia (stress).
- Prueba de rendimiento (volumen y performance).

Pruebas de desempeño

Prueba de estrés (esfuerzo)

Esta prueba implica someter al sistema a grandes cargas o esfuerzos considerables. No confundir con las pruebas de volumen. Un esfuerzo grande es un pico de volúmenes de

datos (normalmente por encima de sus límites) en un corto período de tiempo. No solo volúmenes de datos sino también de usuarios, dispositivos, etc.

Analogía con dactilógrafo:

- Volumen: ver si puede escribir un documento enorme.
- Esfuerzo: ver si puede escribir a razón de 50 palabras x minuto.

Ejemplo: un sistema de conmutación telefónica es sometido a esfuerzo generando un número grande de llamadas telefónicas.

Prueba de volumen

Esta prueba implica someter al sistema a grandes volúmenes de datos. El propósito es mostrar que el sistema no puede manejar el volumen de datos especificado.

Ejemplos:

- A un compilador lo ponemos a compilar un programa absurdamente grande.
- Un simulador de circuito electrónico puede recibir un circuito diseñado con miles de componentes.

Prueba de facilidad de uso

Trata de encontrar problemas en la facilidad de uso.

Algunas consideraciones a tener en cuenta:

- ¿Ha sido ajustada cada interfaz de usuario a la inteligencia y nivel educacional del usuario final y a las presiones del ambiente sobre él ejercidas?
- ¿Son las salidas del programa significativas, no-abusivas y desprovistas de la “jerga de las computadoras”?
- ¿Son directos los diagnósticos de error (mensajes de error) o requieren de un doctor en ciencias de la computación?

Myers presenta más consideraciones. Lo que importa es darse cuenta lo necesario de este tipo de pruebas. Si no se realizan el sistema puede ser inutilizable.

Algunas se pueden realizar durante el prototipado.

Prueba de seguridad

La idea es tratar de generar casos de prueba que burlen los controles de seguridad del sistema.

Ejemplo: diseñar casos de prueba para vencer el mecanismo de protección de la memoria de un sistema operativo.

Una forma de encontrar casos de prueba es estudiar problemas conocidos de seguridad en sistemas similares. Luego mostrar la existencia de problemas parecidos en el sistema bajo test.

Existen listas de descripciones de fallas de seguridad en diversos tipos sistemas.

Prueba de rendimiento

Generar casos de prueba para mostrar que el sistema no cumple con sus especificaciones de rendimiento.

Casos:

- Tiempos de respuesta en sistemas interactivos.
- Tiempo máximo en ejecución de alguna tarea.

Normalmente esto está especificado bajo ciertas condiciones de carga y de configuración del sistema.

Ejemplo: el proceso de facturación no debe durar más de una hora en las siguientes condiciones:

- Se ejecuta en el ambiente descrito en el anexo 1.
- Se ejecuta para 10.000 empleados.

Prueba de configuración

Se prueba el sistema en distintas configuraciones de hardware y software.

Como mínimo las pruebas deben ser realizadas con las configuraciones mínima y máxima posibles.

Prueba de compatibilidad

Son necesarias cuando el sistema bajo test tiene interfaces con otros sistemas.

Se trata de determinar que las funciones con la interfaz no se realizan según especifican los requerimientos.

Prueba de documentación

La documentación del usuario debe ser objeto de inspección controlando su exactitud y claridad (entre otros).

Además, todos los ejemplos que aparezcan en la misma deben ser codificados como casos de prueba. Es importantísimo que al ejecutar estos casos de prueba los resultados sean los esperados.

Un sistema uruguayo tiene en su manual de usuario un ejemplo que no funciona en el sistema.

Prueba de facilidad de instalación

Se prueban las distintas formas de instalar el sistema.

Prueba de recuperación

Se intenta probar que no se cumplen los requerimientos de recuperación.

Para esto se simulan o crean fallas y luego se testea la recuperación del sistema.

Prueba de confiabilidad

El propósito de toda prueba es aumentar la confiabilidad del sistema.

Este tipo de prueba es aquella que se realiza cuando existen requerimientos específicos de confiabilidad del sistema → Se deben diseñar casos de prueba especiales.

Ejemplo: el sistema Bell TSPS dice que tiene un tiempo de detención de 2 horas en 40 años. No se conoce forma de probar este enunciado en meses o en unos pocos años.

De otros casos se puede estimar la validez con modelos matemáticos. Ejemplo: El sistema tiene un valor medio de tiempo entre fallas de 20 horas.

Prueba de aceptación del usuario

Pruebas realizadas por los usuarios, para verificar que el sistema se ajusta a sus requerimientos. Los casos de prueba están basados en la especificación de requerimientos es una técnica de caja negra. Se determina como aceptar un sistema y si es la etapa en que el usuario puede rechazarlo y además es la última oportunidad real de testeo.

A continuación, enumeramos los pasos o actividades del proceso más común:

1. Determinar el rol del usuario.
2. Definir los criterios de aceptación.
3. Desarrollar un plan de aceptación.
4. Ejecutar el plan de aceptación.
5. Determinar decisiones de aceptación.

Recomendaciones a los puntos del plan de aceptación

Definir el rol del usuario

- Asegurar que se involucre en todo momento.
- Identificar criterios de aceptación iniciales, intermedios y finales.
- Planear como y quien aceptara el sistema.
- Planear recursos.
- Preparar el plan de aceptación.

Definir los criterios de aceptación

- Funcionalidad.
- Performance.
- Interfaz.
- Calidad.
- Seguridad.
- Conformidad.

Criticidad de aceptación

- Importancia del sistema.
- Consecuencias de una falla.
- Complejidad del proyecto.
- Riesgo tecnológico.
- Complejidad del ambiente.

Pruebas de regresión

Pruebas orientadas a verificar que, luego de introducido un cambio en el código, la funcionalidad original no ha sido alterada:

- Tengo que probar “lo nuevo y lo viejo”.
- Para esto necesitamos condiciones y casos de prueba “reusables”; si no, esta tarea no puede hacerse.

¿Qué es una prueba de regresión?

El test de regresión es un test funcional de integración.

Se realiza al final de un ciclo de *testing* para verificar que el nuevo *build* o incremento no ha “desarreglado” casos oportunamente probados.

En la regresión se pueden correr todos los casos o seleccionar un subconjunto de ellos.

¿Qué hacer si la prueba de regresión detecta defectos?

Si el test de regresión detecta defectos, se debe:

- Establecer un impacto de la prueba fallada.
- Armar un ciclo de prueba con las pruebas falladas y las pruebas sobre las que impacta.
- Cuando se eliminan los defectos, hacer regresión sobre ese *build*, donde el *build* “posterior” pasa a ser un *build* “anterior”.

Prueba por tipo de sistema

Prueba de instalación

- Su mayor propósito es encontrar errores de instalación.
- Es de mayor importancia si la prueba de aceptación no se realizó en el ambiente de instalación.

Pruebas piloto

- Se pone a funcionar el sistema en producción de forma localizada.
- Menor impacto de las fallas.

Desarrollo para múltiples clientes

- Prueba alfa: realizada por el equipo de desarrollo sobre el producto final.
- Prueba beta: realizada por clientes elegidos sobre el producto final. Se podría decir que Windows está siempre en beta *testing*.

Desarrollo de un sistema que sustituye a otro

Pruebas en paralelo:

- Se deja funcionando el sistema “viejo” mientras se empieza a usar al nuevo.
- Se hacen comparaciones de resultados y se intenta ver que el sistema nuevo funciona adecuadamente.